- *Huge Docker ecosystem.* We frequently need to incorporate other technologies as part of our Rails apps: NGINX, Redis, Postgres, MySQL, Memcached, Elasticsearch, HAProxy, RabbitMQ, Node, and so on. All these and more are already packaged and ready to go with Docker.
- *Simulate production-like environments locally.* We know that how our Rails app performs in development isn't exactly the same as in production. With Docker, you can simulate production scenarios by running your app in multi-node, production-like environments on your local machine.

Docker can also help once you move beyond development. It provides a consistent interface, whether you're running locally, on a continuous integration (CI) server, or deploying to production. Once built, the same image is run at every stage of your continuous integration/delivery pipeline, giving us confidence that our tested application will perform the same in each environment.

If you need to manage and deploy to your own production infrastructure, there are further benefits:

- *Deployment standardization.* Docker provides a standard way of packaging and delivering applications: each part of your app is a container, and each app is a collection of containers. From a DevOps perspective, one Docker app is deployed and managed in the same way as any other.
- *Reliability and resiliency features built in.* Ever been woken at 3 a.m. by a cranky CEO because your app's gone kaput? Docker clusters are self-healing: if an instance dies, new copies of your app will be spawned on the remaining nodes.
- *Reducing infrastructure costs especially at scale.* Containers are much lighter-weight than virtual machines (VMs), allowing resources to be used more efficiently. They also let you scale up the number of containers on a single host rather than spinning up an entire new instance.
- *Room to grow.* If your app is (or becomes) wildly successful, it's good to know that Docker has been battle-tested at massive scale. Google Compute Engine, for example, is built on Docker containers, using Google's open source orchestration tool, Kubernetes.

Who Should Read This Book?

This book is for experienced Rails developers who want to learn how to use Docker. I'm going to assume, throughout the book, that you're proficient at using Rails; this will allow us to focus on learning and applying Docker.